

Python Inheritance

Code-Only Practice Set

Single Inheritance | Multiple Inheritance | Multilevel Inheritance | Method Overriding | super()

No theory here — only coding tasks and "predict the output" questions, so you can practice by actually writing and running Python code. Every question is a real hands-on exercise.

Part	Topic	Question types
1	Single Inheritance	Write code + predict output
2	Multiple Inheritance	Write code + predict output
3	Multilevel Inheritance	Write code + predict output
4	Method Overriding	Write code + predict output
5	super()	Write code + predict output
6	Mixed / Final Practice	Bigger programs, all topics combined

How to use this sheet: For "write code" questions, open your editor and actually type the class and run it — don't just imagine the answer. For "predict the output" questions, write down your guess first, then run the code to check if you were right.

Part 1 – Single Inheritance

1. **[Write Code]** Create a class **Animal** with a method **eat()** that prints "This animal eats food".

Create a class **Dog** that inherits from **Animal** and has its own method **bark()** that prints "The dog barks". Create a Dog object and call both methods.

2. **[Write Code]** Create a class **Person** with attributes **name** and **age**. Create a class **Student** that inherits from **Person** and adds an attribute **roll_number**. Create a Student object and print all three details.

3. **[Predict Output]** Predict the output:

```
class Vehicle:
    def start(self):
        print("Vehicle started")

class Car(Vehicle):
    def drive(self):
        print("Car is driving")
```

```
c = Car()
c.start()
c.drive()
```

4. **[Predict Output]** Predict what happens when this code runs (does it print something, or give an error?):

```
class Vehicle:
    def start(self):
        print("Vehicle started")

class Car(Vehicle):
    def drive(self):
        print("Car is driving")
```

```
v = Vehicle()
v.drive()
```

5. **[Write Code]** Create a class **Shape** with a method **info()** that prints "I am a shape". Create a class **Circle** that inherits from **Shape** and has a method **area(radius)** that returns the area of the circle. Test it with radius = 5.

6. **[Write Code]** Create a class **Animal** with an **__init__(self, name)** that stores the name. Create a class **Dog** that inherits from **Animal** and adds a method **bark()**. Create a Dog object with a name and call **bark()**, then print the dog's name.

7. **[Predict Output]** Predict the output:

```
class Book:
    def __init__(self, title):
        self.title = title

    def show_title(self):
        print("Title:", self.title)

class EBook(Book):
    def show_format(self):
        print("Format: Digital")
```

```
b = EBook("Python Basics")
b.show_title()
b.show_format()
```

8. **[Write Code]** Create a class **BankAccount** with attribute **balance** and a method **show_balance()**. Create a class **SavingsAccount** that inherits from it and adds a method **add_interest()** that increases the balance by 10%. Test it starting with a balance of 1000.

9. **[Write Code]** Create a class **Employee** with attributes **name** and **salary**, and a method **show_salary()**. Create a class **Intern** that inherits from **Employee** and adds a method **show_stipend()** that prints the salary as a stipend instead. Test both methods on an **Intern** object.

Part 2 – Multiple Inheritance

10. **[Write Code]** Create two classes, **Father** with a method **skills()** that prints "Gardening, Cooking", and **Mother** with a method **skills()** that prints "Cooking, Painting". Create a class **Child** that inherits from both. Create a **Child** object and call **skills()**.

11. **[Predict Output]** Predict the output, and explain in one line why that class was chosen:

```
class A:
    def show(self):
        print("Class A")
```

```
class B:
    def show(self):
        print("Class B")
```

```
class C(A, B):
    pass
```

```
obj = C()
obj.show()
```

12. **[Write Code]** Create a class **Flyer** with a method **fly()** that prints "I can fly", and a class **Swimmer** with a method **swim()** that prints "I can swim". Create a class **Duck** that inherits from both, so a **Duck** object can both fly and swim.

13. **[Write Code]** Using the **Duck** class from the question above, write one line of code to print the Method Resolution Order (MRO) of the **Duck** class.

14. **[Write Code]** Create a class **Engine** with a method **start_engine()** and a class **MusicSystem** with a method **play_music()**. Create a class **Car** that inherits from both, and call both methods from a **Car** object.

15. **[Predict Output]** Predict the output:

```
class Walker:
    def move(self):
        print("Walking")
```

```
class Runner:
    def move(self):
        print("Running")
```

```
class Athlete(Runner, Walker):
    pass
```

```
a = Athlete()
a.move()
```

16. **[Write Code]** Create a class **Camera** with a method **take_photo()** and a class **Phone** with a method **make_call()**. Create a class **SmartPhone** that inherits from both and adds its own method **browse_internet()**. Call all three methods from one **SmartPhone** object.

17. [Write Code] Create a class **ChessPlayer** with a method **play_chess()** and a class **Coder** with a method **write_code()**. Create a class **Student** that inherits from both. Create a **Student** object and call both methods, then add a third method of your own, **study()**, only in **Student**.

Part 3 – Multilevel Inheritance

18. [Write Code] Create a class **Animal** with a method **eat()**. Create a class **Bird** that inherits from **Animal** and adds a method **fly()**. Create a class **Parrot** that inherits from **Bird** and adds a method **talk()**. Create a **Parrot** object and call all three methods.

19. [Predict Output] Predict the output:

```
class GrandParent:
    def hobby(self):
        print("Gardening")

class Parent(GrandParent):
    def job(self):
        print("Teacher")

class Child(Parent):
    def game(self):
        print("Cricket")

c = Child()
c.hobby()
c.job()
c.game()
```

20. [Write Code] Create a 3-level chain: **Company** (method **company_info()**) → **Department** (method **department_info()**) → **Employee** (method **employee_info()**). Create an **Employee** object and call all three methods.

21. [Write Code] Create a class **Vehicle** with an **__init__** that stores **brand**. Create a class **Car** that inherits from **Vehicle** and adds **model**. Create a class **SportsCar** that inherits from **Car** and adds **top_speed**. Create a **SportsCar** object with all three details and print them.

22. [Write Code] Using the **SportsCar** class above, write code with **isinstance()** to check if a **SportsCar** object is also considered a **Vehicle**. Print the result.

23. [Predict Output] Predict the output:

```
class Grandfather:
    def surname(self):
        print("Surname: Sharma")

class Father(Grandfather):
    def house(self):
        print("House: Delhi")

class Son(Father):
    def car(self):
        print("Car: SUV")

s = Son()
s.surname()
s.house()
s.car()
```

24. [Write Code] Create a class **LivingThing** with a method **breathe()**. Create a class **Mammal** that inherits from it and adds **walk()**. Create a class **Human** that inherits from **Mammal** and adds **speak()**.

Create a Human object and call all methods in order.

Part 4 – Method Overriding

25. [Write Code] Create a class **Animal** with a method **sound()** that prints "Some generic animal sound". Create a class **Cat** that inherits from **Animal** and overrides **sound()** to print "Meow". Create objects of both classes and call **sound()** on each.

26. [Predict Output] Predict the output:

```
class Employee:
    def salary(self):
        print("Salary is 20000")

class Manager(Employee):
    def salary(self):
        print("Salary is 50000")

e = Employee()
m = Manager()
e.salary()
m.salary()
```

27. [Write Code] Create a class **Shape** with a method **area()** that prints "Area is not defined". Create three classes, **Square**, **Rectangle**, and **Triangle**, that each inherit from **Shape** and override **area()** to calculate and print the correct area for each shape.

28. [Write Code] Create a class **Bank** with a method **interest_rate()** that returns 5. Create a class **SavingsAccount** that inherits from **Bank** and overrides **interest_rate()** to return 7. Print the interest rate from an object of each class.

29. [Predict Output] Predict the output:

```
class Phone:
    def price(self):
        return 10000

class SmartPhone(Phone):
    def price(self):
        return 25000

p1 = Phone()
p2 = SmartPhone()
print(p1.price())
print(p2.price())
```

30. [Write Code] Create a base class **Shape** with a method **area()** that returns 0. Create **Circle** and **Square** classes that override **area()**. Put objects of both classes in a list and use a loop to call **area()** on each one.

31. [Write Code] Create a class **PaymentMethod** with a method **pay(amount)** that prints "Paying {amount} using generic method". Create classes **CreditCard** and **UPI** that inherit from it and override **pay(amount)** with their own message. Call **pay(500)** on each object.

32. [Write Code] Create a class **Notification** with a method **send()** that prints "Sending a notification". Create **EmailNotification** and **SMSNotification** classes that inherit from it and override **send()**. Loop through a list containing one object of each class and call **send()** on all of them.

Part 5 – super()

33. [Write Code] Create a class **Person** with an `__init__` that sets **name** and **age**. Create a class **Student** that inherits from **Person**, uses `super().__init__(name, age)` to set those values, and also sets its own attribute **school_name**. Create a Student object and print all details.

34. [Predict Output] Predict the output:

```
class Parent:
    def greet(self):
        print("Hello from Parent")

class Child(Parent):
    def greet(self):
        super().greet()
        print("Hello from Child")
```

```
c = Child()
c.greet()
```

35. [Write Code] Rewrite a **Cat** class (inheriting from **Animal**, which has a **sound()** method) so that its overridden **sound()** method first calls the parent's **sound()** using `super()`, and then prints "Meow" as an extra line.

36. [Write Code] Create a class **Vehicle** with an `__init__` that sets **brand**. Create a class **Car** that inherits from **Vehicle**, calls `super().__init__(brand)`, and adds its own attribute **model**. Then create a class **SportsCar** that inherits from **Car**, calls `super().__init__(brand, model)`, and adds its own attribute **top_speed**. Print all details of a SportsCar object.

37. [Predict Output] Predict the output:

```
class A:
    def __init__(self):
        print("A's constructor")

class B(A):
    def __init__(self):
        super().__init__()
        print("B's constructor")

class C(B):
    def __init__(self):
        super().__init__()
        print("C's constructor")
```

```
obj = C()
```

38. [Write Code] Create a class **Item** with an `__init__` that sets **name** and **price**, and a method **show()** that prints both. Create a class **DiscountedItem** that inherits from **Item**, calls `super().__init__(name, price)`, adds a **discount** attribute, and overrides **show()** to call `super().show()` first, then print the final price after discount.

39. [Write Code] Create a class **Shape** with a method **describe()** that prints "I am a shape". Create a class **Polygon** that inherits from **Shape**, overrides **describe()** using `super()` to also print "I am a polygon". Create a class **Triangle** that inherits from **Polygon**, overrides **describe()** using `super()` to also print "I am a triangle". Call **describe()** on a Triangle object and check the full output.

Part 6 – Mixed / Final Practice

40. [Write Code] Create a class **Employee** with `__init__(self, name, salary)`. Create a class **Manager** that inherits from **Employee**, uses `super()` to set name and salary, adds a **team_size**

attribute, and overrides a method **details()** to also print the team size.

41. [Write Code] Create classes **Reader** (method **read()**) and **Writer** (method **write()**). Create a class **Editor** that inherits from both using multiple inheritance, and also has its own method **edit()**. Call all three methods on one Editor object.

42. [Write Code] Create a 3-level chain **LivingThing** → **Animal** → **Dog**, where each class adds one new attribute using **super().__init__()**, and the **Dog** class overrides a method called **describe()** that also calls the parent's **describe()** using **super()**.

43. [Predict Output] Predict the output, and figure out the Method Resolution Order it follows:

```
class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        super().show()
        print("B")

class C(A):
    def show(self):
        super().show()
        print("C")

class D(B, C):
    def show(self):
        super().show()
        print("D")
```

```
d = D()
d.show()
```

44. [Write Code] Design a small program for a school system: a class **Person** (name, age), a class **Staff** that inherits from Person and adds **salary**, and a class **Teacher** that inherits from Staff and adds **subject**. Use **super()** at every level, and override a **show_details()** method at each level to print more information as you go down the chain.

45. [Write Code] Create a class **Media** with attributes **title** and a method **play()** that prints "Playing media". Create **Video** and **Audio** classes that inherit from Media and override **play()** using **super()** plus their own extra print line. Then create a class **Podcast** that inherits from **Audio** (multilevel) and overrides **play()** again, also using **super()**. Call **play()** on a Podcast object and check the full chain of output.

46. [Write Code] Create a class **Device** with **power_on()** and **power_off()** methods. Create a class **Speaker** and a class **WifiModule**, both inheriting from Device, each overriding **power_on()** with their own message using **super()**. Create a class **SmartSpeaker** that inherits from both **Speaker** and **WifiModule** (multiple inheritance). Call **power_on()** on a SmartSpeaker object and check which version runs, then print the MRO to confirm.

47. [Predict Output] This code has 4 classes mixing multilevel and overriding with **super()**. Predict the exact output line by line before running it:

```
class Base:
    def info(self):
        print("Base info")

class Middle(Base):
    def info(self):
```

```

        super().info()
        print("Middle info")

class Top(Middle):
    def info(self):
        super().info()
        print("Top info")

t = Top()
t.info()
m = Middle()
m.info()

```

48. [Write Code] Build a mini "vehicle rental" system: a class **Vehicle** with **brand** and a method **rent_price()** returning 500. Create **Bike** and **Car** classes overriding **rent_price()** with different values. Create a class **LuxuryCar** that inherits from **Car** (multilevel), overrides **rent_price()** using **super()** to add an extra charge on top of the Car price. Print the final price for a **LuxuryCar** object.

49. [Write Code] Create a class **Instrument** with a method **play()** that prints "Playing an instrument". Create **Guitar** and **Piano** classes that inherit from it and override **play()**. Create a list with one **Guitar** and one **Piano** object, loop through it, and call **play()** on each.

50. [Predict Output] Predict the output:

```

class A:
    def hello(self):
        print("Hello from A")

class B(A):
    pass

class C(B):
    def hello(self):
        print("Hello from C")

obj1 = B()
obj2 = C()
obj1.hello()
obj2.hello()

```

51. [Write Code] Create a class **Account** with an **__init__** that sets **owner** and **balance**, and a method **withdraw(amount)** that reduces the balance. Create a class **CurrentAccount** that inherits from **Account**, calls **super().__init__(owner, balance)**, adds an **overdraft_limit** attribute, and overrides **withdraw(amount)** so it allows the balance to go negative up to the overdraft limit, using **super().withdraw(amount)** inside it.

Total questions in this sheet: 51

Tip: Do the [Write Code] questions in your editor, not on paper. For [Predict Output] questions, write your guess in a comment above the code, then run it to check yourself.